

Majisa WhatsApp CRM & AI Lead Capture System

Enterprise Technical Documentation &
Operations Guide

Client Name: Majisa Web Solutions
Author: Technical Engineering Team
Date: July 2026
Document Version: v1.0.0 (Release)

1. Executive Summary & Architecture

The **Majisa WhatsApp CRM & AI Lead Capture System** is a full-stack, enterprise-grade system designed to automate lead generation, dynamic package pricing, document proposal creation, and live agent chat management. By integrating directly with the WhatsApp Web interface, the application removes friction from customer acquisition and optimizes the entire sales workflow.

1.1 System Components

- **Frontend Dashboard (React / Vite):** A highly responsive portal for administrative agents featuring real-time KPIs, visual analytics charts, chat threads, customer cards, and custom pricing rule settings.
- **Backend REST API & Worker (Express / Node.js):** Manages auth, logs, CRUD rules, quotation compiler engines, and hosts static attachments.
- **Database (MongoDB / Mongoose):** Relational document store housing persistent customer states, conversation histories, pricing models, audit logs, and notification schemas.
- **WhatsApp Service (Puppeteer / Headless Browser):** Runs a headless Chrome instance to control the WhatsApp Web application protocol, emitting session events and handling incoming messages.

1.2 Communication Protocol

Real-time events (incoming messages, customer typing statuses, read receipts, and system alerts) are synchronized between the server and the admin panel using **Socket.io**. This allows support agents to view active conversations and take over chatbot flows instantly.

2. Core Capabilities & Workflows

2.1 Conversational State Machine

The automated chatbot guides users through a structured requirement gathering questionnaire. The flow is maintained persistently in the database per customer ID via the `ChatState` schema.

Chatbot Questionnaire States

State ID	Target Field Captured	Validation / Action Triggered
<code>WELCOME</code>	N/A (Greeting Menu)	Presents the 6 services, portfolio, and executive-talk options.
<code>ASK_NAME</code>	<code>customer.name</code>	Stores full name; supports navigation back (0).
<code>ASK_COMPANY</code>	<code>customer.company</code>	Stores company name.
<code>ASK_EMAIL</code>	<code>customer.email</code>	Validates email format strictly.
<code>ASK_PHONE</code>	<code>customer.phone</code>	Validates phone format (10 digits).
<code>ASK_CITY</code>	<code>customer.city</code>	Stores customer location.
<code>SELECT_SUB_TYPE</code>	<code>chatState.data.subType</code>	Queries <code>PricingRule</code> for base packages (e.g. Static vs E-commerce).
<code>SELECT_PAGES</code>	<code>chatState.data.pageRange</code>	Gathers required page depth.
<code>SELECT_FEATURES</code>	<code>chatState.data.selectedFeatures</code>	Multi-select features (comma separated list, e.g. 1,3,4) or "skip".

State ID	Target Field Captured	Validation / Action Triggered
SHOW_QUOTATION	N/A (Calculates Invoice)	Generates dynamic PDF and sends it directly to the customer's WhatsApp.

Anti-Loop Loop Guard

To prevent conversational loops and frustration, the chatbot tracks invalid entries. If a user provides an invalid option **3 consecutive times**:

1. The chatbot is programmatically paused (`customer.isBotPaused = true`).
2. The customer's pipeline status is updated to **Talk to Executive**.
3. A high-priority notification (`EXECUTIVE_REQUESTED`) is dispatched via Socket.io to the admin panel dashboard.
4. An polite message is returned to the user stating they are being connected to a human representative.

2.2 NLP-Based Lead Auto-Extraction

When a user sends an unstructured introduction (e.g., in their initial greeting), the system bypasses step-by-step questioning. Using natural language keyword matching and pattern parsing (regex), it extracts key parameters:

- **Email:** Match regular RFC email standards.
- **Phone:** Captures Indian (+91/0) or standard international formats.
- **Company:** Identified using constructs like *"from [Company]"* or inferred from corporate email domains (e.g. `rahul@apex.com` → `Apex`).
- **Service Interest:** Scans text for terms like "wordpress", "app", "marketing", "seo", or "server" and maps them to appropriate service categories.

3. Database Reference Guide

The database consists of collections modeled using Mongoose schemas. Below are details on the core data models.

3.1 Customer Schema (`Customer.js`)

```
// Structure overview of the Customer model
{
  customerId:    { type: String, required: true, unique: true }, // e.g. "9194
0xxxxxxxx@c.us"
  name:         { type: String, default: "" },
  company:      { type: String, default: "" },
  email:        { type: String, default: "" },
  phone:        { type: String, default: "" },
  city:         { type: String, default: "" },
  service:      { type: String, default: "" },
  budget:       { type: String, default: "" },
  timeline:     { type: String, default: "" },
  status:       { type: String, default: "New Lead" }, // New Lead | Talk to
Executive | In Progress | Completed
  source:       { type: String, default: "WhatsApp" },
  quotationSent: { type: Boolean, default: false },
  assignedTo:   { type: String, default: "" },           // Admin Owner ID
  notes:        { type: String, default: "" },
  isBotPaused:  { type: Boolean, default: false },
  isDeleted:    { type: Boolean, default: false }
}
```

3.2 Message Schema (`Message.js`)

```
{
  chat:         { type: Schema.Types.ObjectId, ref: 'Chat', required: true },
  customer:     { type: Schema.Types.ObjectId, ref: 'Customer', required: true },
  sender:       { type: String, enum: ['CUSTOMER', 'ADMIN', 'BOT'], required: true },
  message:      { type: String, required: true },           // Text content or st
```

```
atic file path
  type:          { type: String, enum: ['TEXT', 'IMAGE', 'PDF', 'VIDEO', 'AUDIO', 'DOCUMENT'] },
  status:        { type: String, enum: ['SENT', 'DELIVERED', 'READ'] }
}
```

3.3 Pricing Rule Schema (`PricingRule.js`)

This schema stores dynamic pricing configuration. Categories determine how options render to the chatbot:

- `SERVICE`: Major service types (e.g., Website Development, Mobile Apps).
- `PACKAGE`: Base package subtypes mapped to specific services (e.g. E-Commerce package).
- `PAGE_RANGE`: Base cost margins and pricing-per-extra-page.
- `FEATURE`: Optional plug-and-play features (e.g., Payment Gateway Integration, Chatbot).

4. API Reference Manual

All routes are versioned and prefixes are mapped to `/api/v1`. Access to administrative routes (chats, settings, reports) requires authorization via JWT authentication cookies.

4.1 Endpoints List

Authentication & Profile

Method	Path	Description
POST	<code>/api/v1/auth/register</code>	Creates a new admin login credential.
POST	<code>/api/v1/auth/login</code>	Validates password and issues secure JWT HttpOnly cookie.
POST	<code>/api/v1/auth/logout</code>	Clears the session cookie.
GET	<code>/api/v1/auth/me</code>	Returns details of currently authenticated admin.

Live Chat Console

Method	Path	Description
GET	<code>/api/v1/chats</code>	Lists active chat rooms sorted by last activity.
GET	<code>/api/v1/chats/:customerId/messages</code>	Returns a paginated list of logs/messages for that thread.
POST	<code>/api/v1/chats/send</code>	Dispatches an outgoing WhatsApp message on behalf of admin.
POST	<code>/api/v1/chats/send-media</code>	Uploads base64 files and sends them as media attachments.
PUT	<code>/api/v1/chats/:chatId/read</code>	Resets unread count in dashboard.

Method	Path	Description
DELETE	/api/v1/chats/:chatId	Permanently clears message histories and chat session records.

Leads & Customers Directory

Method	Path	Description
GET	/api/v1/customers	Returns customer listing (with filtering and text search).
GET	/api/v1/customers/:id	Fetches specific customer profile records.
PUT	/api/v1/customers/:id	Modifies details, updates assignedOwner, adds notes, or toggles bot.
DELETE	/api/v1/customers/:id	Performs a soft-delete (sets <code>isDeleted = true</code>).

5. Deployment & Operations Guide

5.1 Environment Configuration

The server relies on the following configurations. Ensure a `.env` file is configured in the `/server` directory:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/majisa_whatsapp_crm
JWT_SECRET=your_jwt_signing_key_secret
JWT_EXPIRES_IN=7d
CLIENT_URL=http://localhost:5173
```

5.2 Local Execution

To run the application locally without Docker:

1. Start MongoDB

Ensure that a local MongoDB instance is running on port 27017.

2. Run Backend Server

```
cd server
npm install
npm run dev
```

3. Run Frontend Portal

```
cd client
npm install
npm run dev
```

5.3 Docker Compose Deployment

In a production scenario, use the root level `docker-compose.yml`. This binds together the MongoDB, the Express backend, and the React frontend static server (which runs behind an Nginx gateway routing traffic on port 80).

```
# Compile and boot all services in detached mode
docker-compose up --build -d
```

5.4 Backup & Verification Script

A utility script (`server/src/scripts/backup_verify.js`) is included to test database disaster recovery. Running this script copies database states to `./backups`, spins up a temporary restore DB, and checks schema count parity. Run it via:

```
cd server
node src/scripts/backup_verify.js
```

5.5 Automated Tests

Automated regression tests can be run inside each component folder:

- **Backend Tests (Jest):** `cd server && npm test`
- **Frontend Tests (Vitest):** `cd client && npm test`

Majisa Web Solutions CRM © 2026. Confidential Documentation. All Rights Reserved.